



ΨΗΦΙΑΚΑ ΣΥΣΤΗΜΑΤΑ ΗΩ ΣΕ ΧΑΜΗΛΑ ΕΠΙΠΕΔΑ ΛΟΓΙΚΗΣ Ι

ΕΡΓΑΣΙΑ ΕΞΑΜΗΝΟΥ

ΜΑΧΑΙΡΑΣ ΣΤΑΥΡΟΣ – ΑΕΜ: 10046



ΤΜΗΜΑ ΗΛΕΚΤΡΟΛΟΓΩΝ ΜΗΧΑΝΙΚΩΝ & ΜΗΧΑΝΙΚΩΝ
ΥΠΟΛΟΓΙΣΤΩΝ
ΑΡΙΣΤΟΤΕΛΕΙΟ ΠΑΝΕΠΙΣΤΗΜΙΟ ΘΕΣΣΑΛΟΝΙΚΗΣ
ΑΚΑΔΗΜΑΪΚΟ ΕΤΟΣ: 2023-2024
21 ΙΑΝΟΥΑΡΙΟΥ 2023

Πίνακας Περιεχομένων

Εισαγωγή.....	2
Άσκηση 1	3
Άσκηση 2	4
Άσκηση 3	6
Άσκηση 4	7
Άσκηση 5	10
Άσκηση 5 - Alternate Version.....	17
Προβλήματα.....	20

Εισαγωγή

Το παρόν έγγραφο αποτελεί αναφορά της εξαμηνιαίας εργασίας στα πλαίσια του μαθήματος “Ψηφιακά Κυκλώματα Hardware σε Χαμηλά Επίπεδα Λογικής Ι”. Στα πλαίσια της εργασίας υλοποιείται ένα σύστημα επεξεργαστή RISC-V ξεκινώντας από την αριθμητική λογική μονάδα (Arithmetic Logic Unit – ALU, Άσκηση 1). Στη συνέχεια, κατασκευάζεται ένα σύστημα αριθμομηχανής που χρησιμοποιεί την υλοποίηση της ALU, με σκοπό να ελεγχθεί και μέσω testbench η ορθή λειτουργία της μονάδας (Άσκηση 2). Η εργασία προχωράει με την υλοποίηση ενός αρχείου καταχωρητών (Άσκηση 3) και ολοκληρώνεται με την υλοποίηση της διαδρομής δεδομένων (Datapath, Άσκηση 4), καθώς και την μονάδας ελέγχου (Control Unit, Άσκηση 5) μαζί με το testbench για την λειτουργία ολόκληρου του συστήματος του επεξεργαστή (Άσκηση 5).

Άσκηση 1

Σκοπός της άσκησης αυτός είναι η δημιουργία της αριθμητική λογικής μονάδας του επεξεργαστή με θύρες εισόδου και εξόδου 32-bit. Η προς δημιουργία ALU είναι υπεύθυνη για εννιά πράξεις, πρόσθεση (ADD), αφαίρεση (SUB), λογική AND, λογική OR, λογική XOR, μικρότερο από (Set Less Than – SLT), λογική ολίσθηση δεξιά (Shift Right Logic – SRL), λογική ολίσθηση αριστερά (Shift Left Logic – SLL) και αριθμητική ολίσθηση δεξιά (Shift Right Arithmetic – SRA). Ο συνολικός αριθμός των πράξεων είναι εννιά, οπότε το σήμα επιλογής που χρησιμοποιείται είναι 4-bit.

Η υλοποίηση του module “alu” ακολουθεί την προτεινόμενη από την αναφορά σειρά, ξεκινώντας από την αρχικοποίηση των εννιά προβλεπόμενων παραμέτρων για το σήμα “alu_op” της ALU. Έπειτα ακολουθεί η ανάθεση του αποτελέσματος της εκάστοτε πράξης που διενεργεί η μονάδα, με σεβασμό στο σήμα “alu_op”, όπως υποδεικνύεται από τον σχετικό πίνακα (αναφορά Σελίδες 3 & 4). Όλα τα παραπάνω παρατίθενται στα Πλαίσια 1 & 2.

Πλαίσιο 1: ALU - Ορισμός Παραμέτρων

```
parameter[3:0] ALUOP_AND = 4'b0000; //Logic AND
parameter[3:0] ALUOP_OR = 4'b0001; //Logic OR
parameter[3:0] ALUOP_ADD = 4'b0010; //Addition
parameter[3:0] ALUOP_SUB = 4'b0110; //Subtraction
parameter[3:0] ALUOP_SLT = 4'b0111; //Set Less than
parameter[3:0] ALUOP_SRL = 4'b1000; //Logic Shift Right
parameter[3:0] ALUOP_SLL = 4'b1001; //Logic Shift Left
parameter[3:0] ALUOP_SRA = 4'b1010; //Arithmetic Shift Right
parameter[3:0] ALUOP_XOR = 4'b1101; //Logic XOR
```

Πλαίσιο 2: ALU - Ανάθεση Αποτελέσματος

```
assign result =
    (alu_op == ALUOP_AND) ? op1 & op2:
    (alu_op == ALUOP_OR) ? op1 | op2:
    (alu_op == ALUOP_ADD) ? op1 + op2:
    (alu_op == ALUOP_SUB) ? op1 - op2:
    (alu_op == ALUOP_SLT) ? ((-op1 < -op2) ? 32'b1 : 32'b0):
    (alu_op == ALUOP_SRL) ? op1 >> op2[4:0]:
    (alu_op == ALUOP_SLL) ? op1 << op2[4:0]:
    (alu_op == ALUOP_SRA) ? ~((-op1 >>> op2[4:0])):
    (alu_op == ALUOP_XOR) ? op1 ^ op2:
    32'hxxxxxxxx;
assign zero = (result == 32'b0) ? 1'b1 : 1'b0;
```

Άσκηση 2

Στην άσκηση αυτή σχεδιάζεται ένα κύκλωμα αριθμομηχανής με απώτερο στόχο τον έλεγχο της ορθής λειτουργίας της ALU. Για τον σκοπό αυτό δημιουργείται ένας συσσωρευτής 16-bit για να κρατάει την τρέχουσα τιμή της αριθμομηχανής καθώς εισάγονται και τρία buttons για να ρυθμίσουν κατάλληλα τη δημιουργία του σήματος επιλογής “alu_op”. Τα τρία κουμπιά (“btnl”, “btnc”, “btnr”) συνδυάζονται κατάλληλα ώστε να παραχθούν μεμονωμένα τα τέσσερα bit που απαρτίζουν το σήμα επιλογής της ALU.

Η υλοποίηση του module “decoder” είναι η αποκρυπτογράφηση των λογικών μονοπατιών όπως αυτά ορίζονται στα Σχήματα 2, 3, 4 & 5 της αναφοράς για τον υπολογισμό του σήματος “alu_op”. Η υλοποίησή τους σε Verilog παρουσιάζεται στο Πλαίσιο 3.

Πλαίσιο 3: Decoder - Υπολογισμός “alu_op”

```
assign alu_op[0] = (~btnr & btnl) | ((btnl ^ btnc) & btnr);
assign alu_op[1] = (btnr & btnl) | (~btnl & ~btnc);
assign alu_op[2] = ((btnr & btnl) | (btnr ^ btnl)) & ~btnc;
assign alu_op[3] = ((~btnr & btnc) | (btnr ^ btnc)) & btnl;
```

Το module της αριθμομηχανής “calc” υλοποιήθηκε με σεβασμό στις αναλυτικές οδηγίες που αναγράφονται στην αναφορά καθώς και με βάση το διάγραμμα ροής της αριθμομηχανής που απεικονίζεται στο Σχήμα 1 της αναφοράς. Σύμφωνα με αυτό, χρησιμοποιείται ως πρώτο σήμα εισόδου της αριθμομηχανής η επέκταση προσήμου ενός συσσωρευτή στα 32-bit, ενώ ως δεύτερο η επέκταση προσήμου ενός διακόπτη (“sw”) 16-bit στα 32-bit. Ως σήμα ελέγχου της ALU, σήμα επιλογής της εκάστοτε πράξης, χρησιμοποιείται το “alu_op” η δημιουργία του οποίου έχει περιγραφεί παραπάνω. Όπως υποδεικνύεται από την εκφώνηση, για την επαναφορά της αριθμομηχανής χρησιμοποιείται το κουμπί “btneu”, ενώ για την ενημέρωση του συσσωρευτή χρησιμοποιείται το κουμπί “btnd”. Παραλείποντας τις περιττές δηλώσεις μεταβλητών, ο κώδικας του module παρατίθεται στο Πλαίσιο 4.

Πλαίσιο 4: Calc - Υπολογισμός Τιμών Εξόδου & Συσσωρευτή

```
assign op1_0 = {{16{accumulator[15]}}, accumulator};
assign op2_0 = {{16{sw[15]}}, sw};
decoder d1 (.alu_op(alu_op0), .btnl(btnl), .btnr(btnr), .btnc(btnc));
alu a1 (.zero(z0), .result(res0), .op1(op1_0), .op2(op2_0), .alu_op(alu_op0));
always @(posedge clk or posedge btneu)
begin
    if(btneu)
        accumulator = 16'b0;
    if(btnd)
        accumulator = res0[15:0];
end
assign led = accumulator;
```

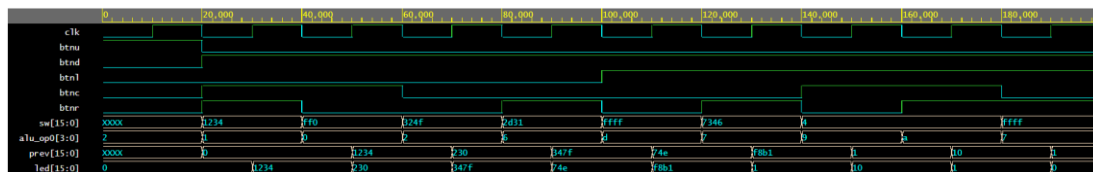
Για τον έλεγχο της ορθής λειτουργίας της αριθμομηχανής και κατά συνέπεια της ALU δημιουργήθηκε το module “calc_tb”. Για την υλοποίηση του testbench χρησιμοποιείται timescale 1ns/1ps, ενώ το ρολόι ορίζεται με περίοδο 20ns και duty cycle 50%. Σε κάθε κύκλο ρολογιού αλλάζουν οι τιμές των “btnr”, “btnc”, “btnl” και “sw” με σκοπό να παράγεται κάθε φορά το επιθυμητό αποτέλεσμα. Οι προς

διερεύνηση τιμές, καθώς και τα αναμενόμενα αποτελέσματα παρατίθενται στον Πίνακα 1 ενώ το διάγραμμα χρονισμού της άσκησης παρατίθεται στο Διάγραμμα 1. Οι ουσιαστικοί κύκλοι ενδιαφέροντος είναι δέκα και για λόγους πληρότητας προστίθενται ένας ακόμη, χωρίς να υπάρχει αλλαγή στις εισόδους της μονάδας.

Πίνακας 1: Είσοδοι & Αναμενόμενες Τιμές Διαγράμματος Χρονισμού - ALU

btnl, btnc, btnr (btnc for reset)	sw	previous value	expcted result
0, 1, 1	xxxx	xxxx	0x0
0, 1, 1	0x1234	0x0	0x1234
0, 1, 0	0x0ff0	0x1234	0x0230
0, 0, 0	0x324f	0x0230	0x347f
0, 0, 1	0x2d31	0x347f	0x074e
1, 0, 0	0xffff	0x074e	0xf8b1
1, 0, 1	0x7346	0xf8b1	0x0001
1, 1, 0	0x0004	0x0001	0x0010
1, 1, 1	0x0004	0x0010	0x0001
1, 0, 1	0xffff	0x0001	0x0000

Διάγραμμα 1: Διάγραμμα Χρονισμού - ALU



Άσκηση 3

Στην άσκηση αυτή κατασκευάζεται το αρχείο καταχωρητών που χρησιμοποιείται από το σύστημα επεξεργαστή RISC-V. Το αρχείο καταχωρητών είναι υψίστης σημασίας καθώς σχεδόν όλες οι εντολές είτε διαβάζουν από αυτό είτε γράφουν σε αυτό. Το αρχείο καταχωρητών που δημιουργείται είναι πλάτους 32×32-bits.

Για το module “regfile” ο κώδικας υλοποιείται όπως υποδεικνύεται στην αναφορά της εργασίας, ξεκινώντας από την αρχικοποίηση του αρχείου καταχωρητών με ένα initial block και μία επαναληπτική διαδικασία for. Ύστερα, με την χρήση ενός always block διαβάζονται οι τιμές των καταχωρητών σε κάθε έξοδο και εγγράφονται δεδομένα στην κατάλληλη είσοδο. Ιδιαίτερη προσοχή επιδεικνύεται στην περίπτωση όπου η διεύθυνση εγγραφής συμπίπτει με μία εκ των διευθύνσεων ανάγνωσης. Στην περίπτωση αυτή και για να μην υπάρξει σύγχυση μεταξύ των δεδομένων εγγραφής και ανάγνωσης, γίνεται εξ’ αρχής η ανάγνωση και μετατίθεται η εγγραφή των δεδομένων για τον επόμενο κύκλο ρολογιού. Παραλείποντας τις περιττές δηλώσεις και τις αρχικοποιήσεις, η διαδικασία εγγραφής και ανάγνωσης παρατίθεται στο Πλαίσιο 5.

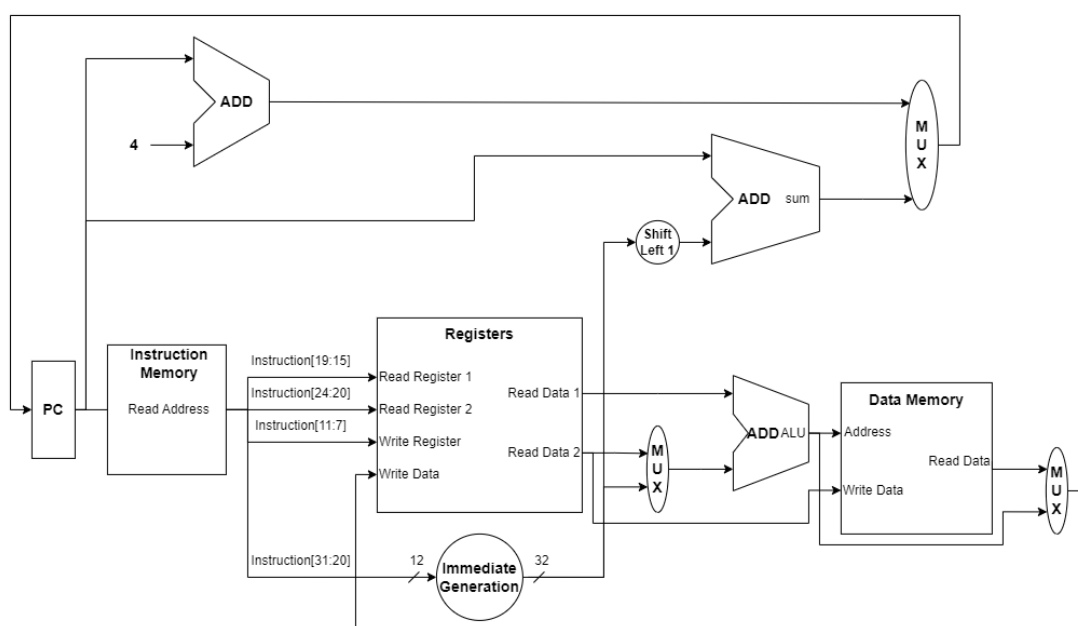
Πλαίσιο 5: RegFile - Διαδικασία Ανάγνωσης & Εγγραφής

```
//basically, the writing process when the reading and writing addresses coincide  
is executed at the next duty cycle.  
parameter reg_length = 32;  
reg [reg_length-1:0] reg_array [reg_length-1:0];  
always @(clk)  
begin  
    readData1 = reg_array[readReg1];  
    readData2 = reg_array[readReg2];  
    if(write)  
    begin  
        if (writeReg==readReg1 || writeReg==readReg2)  
            reg_array[writeReg] <= writeData;  
        else  
            reg_array[writeReg] = writeData;  
        end  
    end  
end
```

Άσκηση 4

Στην άσκηση αυτή κατασκευάζεται το μονοπάτι δεδομένων (Datapath) του συστήματος επεξεργαστή, το οποίο υποστηρίζει εντολές R-type, I-type, S-type και B-type. Συγκεκριμένα, υποστηρίζει τις εντολές ADD, SUB, AND, OR, XOR, SLT, SRL και SRA που είναι R-type (register to register), τις ADDI, ANDI, ORI, XORI, SLTI, SLLI, SRLI, και SRAI που είναι I-type (ALU immediate), τις LW και SW που είναι S-type (load & store word) και την BEQ που είναι B-type (branch). Για την σύνθεση του datapath χρησιμοποιούνται οι υλοποιήσεις της αριθμητικής λογικής μονάδας της Άσκησης 1 και του αρχείου καταχωρητών της Άσκησης 3, αντίστοιχα. Στο Διάγραμμα 2 παρουσιάζεται το προς κατασκευή μονοπάτι δεδομένων που ζητείται να κατασκευαστεί στην άσκηση αυτή. Εξάιρεση αποτελούν η μνήμη εντολών και η μνήμη δεδομένων που δίνονται έτοιμες και χρησιμοποιούνται απευθείας στο testbench της Άσκησης 5.

Διάγραμμα 2: Μονοπάτι Δεδομένων - Datapath



Το module του μονοπατιού δεδομένων “datapath” έχει ως στόχο την λήψη μίας εντολής (instruction), την αποκωδικοποίηση της, αναγνωρίζοντας τον τύπο και τα πεδία της, την ρύθμιση των τελεστών της ALU και εν τέλη την τροφοδότηση των σημάτων ελέγχου (η μονάδα ελέγχου υλοποιείται στην Άσκηση 5) όπου αυτά είναι απαραίτητα.

Το πρώτο κυκλωματικό στοιχείο είναι το PC (Program Counter) που περιέχει τη διεύθυνση μνήμης εντολών, υποδεικνύοντας την θέση στην μνήμη στην οποία βρίσκεται η τρέχουσα εντολή. Η τιμή του ανανεώνεται με το σήμα ελέγχου “LoadPC”, παίρνοντας είτε την τιμή “PC+4” είτε την τιμή “PC+branch_offset”, γεγονός που καθορίζεται μέσω του σήματος επιλογής “PCSrc”. Η δημιουργία των σημάτων αυτών υλοποιείται στην Άσκηση 5. Ο κώδικας που χρησιμοποιείται για την ανανέωση του PC, χωρίς τις αρχικοποιήσεις, παρατίθεται στο Πλαίσιο 6.

Πλαίσιο 6: Datapath - Ανανέωση PC

```
reg [31:0] branch_offset
always @(posedge clk or posedge rst)
begin
    if (rst)
        PC = INITIAL_PC;
    else if (LoadPC)
    begin
        if (PCSrc)
            PC <= PC+branch_offset;
        else
            PC <= PC+32'b0100;
    end
end
```

Δεύτερο στοιχείο είναι η μνήμη εντολών (Instruction Memory), που όπως προαναφέρθηκε δίνεται έτοιμη, και συνεπώς δεν ορίζεται στην άσκηση αυτή. Χρησιμοποιείται, όμως, στην επόμενη άσκηση.

Τρίτο στοιχείο είναι το αρχείο καταχωρητών (Register File), το οποίο έχει δημιουργηθεί στην προηγούμενη άσκηση. Για το μονοπάτι δεδομένων αρκεί η δημιουργία ενός στιγμιότυπου του module “datapath”.

Το τέταρτο στοιχείο είναι το κύκλωμα παραγωγής του σήματος “immediate” (Immediate Generator). Το σήμα αυτό δημιουργείται για εντολές τύπου I, S και B. Για τις πέντε από τις οκτώ εντολές I-type δίνεται από τα 12 MSB του σήματος “instr” (“instr[31:20]”). Για να εξακριβωθεί αν μία εντολή είναι I-type γίνεται έλεγχος του “opcode” (“instr[6:0]”) που πρέπει να έχει την τιμή “0010011” και για να προσδιοριστεί το ότι μία τέτοια εντολή ορίζεται το σήμα “immediate” πρέπει το “funct3” (“instr[14:12]”) να παίρνει μία εκ των τιμών “000”, “010”, “100”, “110” και “111”. Οι εντολές S-type είναι δύο. Για την LW (load word) το “immediate” δίνεται από τα 12 MSB του “instr” και προσδιορίζεται με “opcode” ίσο με “0000011” και “funct3” ίσο με “010”. Για την SW (store word) το “immediate” δίνεται από τα 7 MSB του “instr” και τα 5-bit που βρίσκονται ενδιάμεσα στο “opcode” και το “funct3”, δηλαδή ορίζεται ως {“instr[31:25]”, “instr[11:7]”}, και προσδιορίζεται με “opcode” ίσο με “0100011” και “funct3” ίσο με “010”. Τέλος για εντολές B-type και συγκεκριμένα την BEQ το “immediate” ορίζεται ως {“instr[31]”, “instr[7]”, “instr[30:25]”, “instr[11:8]”} και προσδιορίζεται έχοντας “opcode” ίσο με “1100011” και “funct3” ίσο με “000”. Μετά την δημιουργία του σήματος “immediate” γίνεται επέκταση προσήμου στα 32-bit καθώς και, μόνο για τις εντολές BEQ, ολίσθηση αριστερά κατά 1. Η παραπάνω διαδικασία περιλαμβάνει τον υπολογισμό του “branch_offset”, η οποία και έχει νόημα μόνο όταν η εντολή που εκτελείται είναι η BEQ. Η παραπάνω διαδικασία περιγράφεται σε μορφή κώδικα στο Πλαίσιο 7.

Το πέμπτο στοιχείο του μονοπατιού είναι η ALU για την οποία αρκεί η δημιουργία ενός στιγμιότυπου του module “alu” που υλοποιήθηκε στην Άσκηση 1. Η επιλογή του σήματος που δίνεται στην δεύτερη είσοδο της μονάδας γίνεται μέσω του σήματος ελέγχου “ALUSrc” και υλοποιείται με μία απλή εντολή “assign”.

Πλαίσιο 7: Datapath - Παραγωγή Σήματος "immediate"

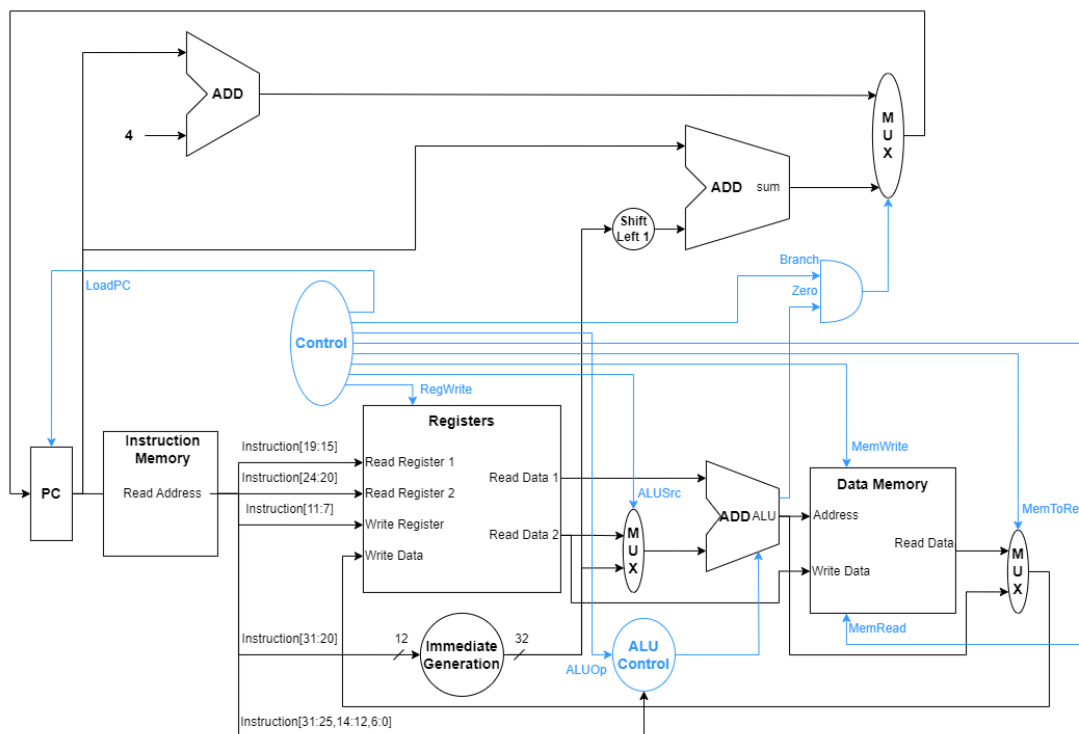
```
reg [31:0] branch_offset,imm;
reg [11:0] imm_temp;
always @(instr)
begin
    if ((instr[6:0]==7'b0010011 && (instr[14:12]==3'b000 || instr[14:12]==3'b010
|| instr[14:12]==3'b100 || instr[14:12]==3'b110 || instr[14:12]==3'b111)) ||
(instr[6:0]==7'b0000011 && instr[14:12]==3'b010))
        begin
            imm_temp = instr[31:20];
            imm = {{20{imm_temp[11]}},imm_temp};
        end
    else if (instr[6:0]==7'b0100011 && instr[14:12]==3'b010)
        begin
            imm_temp = {instr[31:25],instr[11:7]};
            imm = {{20{imm_temp[11]}},imm_temp};
        end
    else if (instr[6:0]==7'b1100011 && instr[14:12]==3'b000)
        begin
            imm_temp = {instr[31],instr[7],instr[30:25],instr[11:8]};
            imm = {{20{imm_temp[11]}},imm_temp};
            branch_offset = imm<<1;
        end
    end
end
```

Το έκτο και τελευταίο κομμάτι του μονοπατιού είναι η μνήμη δεδομένων (Data Memory) μαζί με τη λογική εγγραφής προς τα πίσω. Η μνήμη δεδομένων δίνεται έτοιμη, ενώ η επιλογή του σήματος που οδηγείται για εγγραφή δίνεται με την βοήθεια ενός σήματος ελέγχου "MemToReg" και υλοποιείται με μία απλή εντολή "assign".

Άσκηση 5

Στην άσκηση αυτή κατασκευάζεται η μονάδα ελέγχου. Η μονάδα ελέγχου αποτελεί ένα ελεγκτή πολλών κύκλων που εκτελεί την κάθε εντολής σε πέντε κύκλους ρολογιού. Σκοπός της άσκησης είναι να διαχειριστεί και να ορίσει τα σήματα ελέγχου “ALUCtrl”, “ALUSrc”, “MemRead”, “MemWrite”, “MemToReg”, “RegWrite”, “LoadPC” και “PCSrc”, με την λειτουργικότητα που εμφανίζουν στο Διάγραμμα 3, καθώς και να δημιουργήσει μία μηχανή πεπερασμένων καταστάσεων (Finite State Machine – FSM) σε ένα ενιαίο module “multicycle”.

Διάγραμμα 3: Μονάδα Ελέγχου - Control Unit



Σε πρώτο στάδιο υλοποιείται η διαδικασία επιλογής του 4-bit σήματος ελέγχου “ALUCtrl”. Το σήμα αυτό είναι υπεύθυνο για την επιλογή της πράξης εκείνης που θα εκτελεστεί από την ALU. Για τις εντολές S-type η ALU εκτελεί πρόσθεση, για τις B-type αφαίρεση, ενώ για τις υπόλοιπες εντολές εκτελείται κανονικά η προβλεπόμενη πράξη. Για να γίνει αποκρυπτογράφηση της εντολής χρειάζονται το “opcode” (“instr[6:0]”), το “funct3” (“instr[14:12]”) και το “funct7” (“instr[31:25]”). Γενικότερα, πρόσθεση (ADD) εκτελείται στις εντολές LW, SW, ADDI και ADD, αφαίρεση (SUB) εκτελούν οι BEQ και SUB. Λογικό και (AND) εκτελούν οι ANDI και AND, λογικό ή (OR) εκτελούν οι ORI και OR, ενώ αποκλειστικό διάζευξη (XOR) εκτελούν οι XORI και XOR. Λογική ολίσθηση προς τα δεξιά (SRL) εκτελούν οι SRLI και SRL, λογική ολίσθηση προς τα αριστερά (SLL) εκτελούν οι SLLI και SLL, ενώ αριθμητική ολίσθηση προς τα δεξιά (SRA) εκτελούν οι SRAI και SRA. Τέλος, μικρότερο από (SLT) εκτελούν οι SLTI και SLT. Οι τιμές των “opcode”, “funct3” και “funct7” για κάθε λειτουργία ξεχωριστά παρουσιάζονται στον Πίνακα 2. Η δημιουργία του σήματος “ALUCtrl”, όπως περιεγράφηκε παραπάνω, με σεβασμό στον τύπο της εντολής παρατίθεται στο Πλαίσιο 8.

Πίνακας 2: Multicycle - Καθορισμός "opcode", "funct3", "funct7" για "ALUCtrl"

Instruction	Operation	Opcode	Funct3	Funct7
LW	ADD	0000011	010	
SW		0100011		
ADDI		0010011	000	0000000
ADD		0110011		
BEQ	SUB	1100011		0100000
SUB		0110011		
ANDI	AND	0010011	111	0000000
AND		0110011		
ORI	OR	0010011	110	0000000
OR		0110011		
XORI	XOR	0010011	100	0000000
XOR		0110011		
SRLI	SRL	0010011	101	0000000
SRL		0110011		
SLLI	SLL	0010011	001	0000000
SLL		0110011		
SRAI	SRA	0010011	101	0100000
SRA		0110011		
SLTI	SLT	0010011	010	0000000
SLT		0110011		

Σε δεύτερο στάδιο κατασκευάζεται το σήμα "ALUSrc" που είναι υπεύθυνο για την επιλογή του σήματος στην δεύτερη είσοδο της ALU. Γενικότερα, άμεσα δεδομένα "immediate" χρειάζονται οι εντολές S-type και I-type (όχι όλες) των οποίων τα "opcode" και "funct3" προσδιορίζονται στον Πίνακα 2. Με βάση λοιπόν τα παραπάνω χαρακτηριστικά στο Πλαίσιο 9 παρατίθεται η διαδικασία δημιουργίας του σήματος "ALUSrc".

Σε τρίτο στάδιο κατασκευάζεται το σήμα "PCSrc" το οποίο προσδιορίζει το αν προκύπτει ή όχι διακλάδωση και κατά συνέπεια υποδεικνύει στο PC τον τρόπο με τον οποίο οφείλει να ανανεώσει την τιμή του. Διακλάδωση προκύπτει μονάχα σε εντολές B-type, συγκεκριμένα της BEQ, τα "opcode" και "funct3" της οποίας έχουν ήδη παρουσιασθεί στον Πίνακα 2. Η δημιουργία του σήματος αυτού παρατίθεται στο Πλαίσιο 10.

Τα υπόλοιπα σήματα ελέγχου εξαρτώνται από την κατάσταση στην οποία βρίσκεται το σύστημα του επεξεργαστή. Για τον συγκεκριμένο επεξεργαστή παρατηρούνται πέντε διακριτές καταστάσεις με την ακόλουθη σειρά: instruction fetch (IF), instruction decode (ID), execution (EX), memory (MEM) και write back (WB), με κάθε μία από αυτές να καταλαμβάνει και από ένα κύκλο ρολογιού, συνολικά μαζί πέντε κύκλους. Υπάρχει το ενδεχόμενο, σε περιπτώσεις που δεν απαιτείται αποθήκευση ή ανάκτηση δεδομένων προς και από την μνήμη, να παραλείπεται η κατάσταση MEM και συνεπώς η όλη διαδικασία να πραγματοποιείται σε τέσσερις κύκλους ρολογιού. Ένα απλό σχηματικό του FSM απεικονίζεται στο Διάγραμμα 4.

Κάθε κατάσταση του FSM επηρεάζεται από διαφορετικά σήματα ελέγχου. Η πρώτη κατάσταση είναι η IF για την οποία απαιτείται να έχει δοθεί στο σήμα "LoadPC" η

Πλαίσιο 8: Multicycle - Δημιουργία "ALUCtrl"

```
always @(instr)
begin
    //SUB Operation (BEQ, SUB)
    if ((instr[6:0]==7'b1100011 && instr[14:12]==3'b000) ||
(instr[6:0]==7'b0110011 && instr[14:12]==3'b000 && instr[31:25]==7'b0100000))
        ALUCtrl = 4'b0110;
    //ADD Operation (ADD, LW, SW)
    else if (((instr[6:0]==7'b0000011 || instr[6:0]==7'b0100011) &&
instr[14:12]==3'b010) || (instr[6:0]==7'b0010011 && instr[14:12]==3'b000) ||
(instr[6:0]==7'b0110011 && instr[14:12]==3'b000 && instr[31:25]==7'b0000000))
        ALUCtrl = 4'b0010;
    //AND Operation (AND, ANDI)
    else if ((instr[6:0]==7'b0010011 && instr[14:12]==3'b111) ||
(instr[6:0]==7'b0110011 && instr[14:12]==3'b111 && instr[31:25]==7'b0000000))
        ALUCtrl = 4'b0000;
    //OR Operation (OR, ORI)
    else if ((instr[6:0]==7'b0010011 && instr[14:12]==3'b110) ||
(instr[6:0]==7'b0110011 && instr[14:12]==3'b110 && instr[31:25]==7'b0000000))
        ALUCtrl = 4'b0001;
    //XOR Operation (XOR, XORI)
    else if ((instr[6:0]==7'b0010011 && instr[14:12]==3'b100) ||
(instr[6:0]==7'b0110011 && instr[14:12]==3'b100 && instr[31:25]==7'b0000000))
        ALUCtrl = 4'b1101;
    //SLT Operation (SLT)
    else if ((instr[6:0]==7'b0010011 && instr[14:12]==3'b010) ||
(instr[6:0]==7'b0110011 && instr[14:12]==3'b010 && instr[31:25]==7'b0000000))
        ALUCtrl = 4'b0111;
    //SRL Operation (SRL, SRLI)
    else if ((instr[6:0]==7'b0010011 && instr[14:12]==3'b101 &&
instr[31:25]==7'b0000000) || (instr[6:0]==7'b0110011 && instr[14:12]==3'b101 &&
instr[31:25]==7'b0000000))
        ALUCtrl = 4'b1000;
    //SLL Operation (SLL, SLLI)
    else if ((instr[6:0]==7'b0010011 && instr[14:12]==3'b001 &&
instr[31:25]==7'b0000000) || (instr[6:0]==7'b0110011 && instr[14:12]==3'b001 &&
instr[31:25]==7'b0000000))
        ALUCtrl = 4'b1001;
    //SRA Operation (SRA, SRAI)
    else if ((instr[6:0]==7'b0010011 && instr[14:12]==3'b101 &&
instr[31:25]==7'b0100000) || (instr[6:0]==7'b0110011 && instr[14:12]==3'b101 &&
instr[31:25]==7'b0100000))
        ALUCtrl = 4'b1010;
end
```

Πλαίσιο 9: Multicycle - Καθορισμός "ALUSrc"

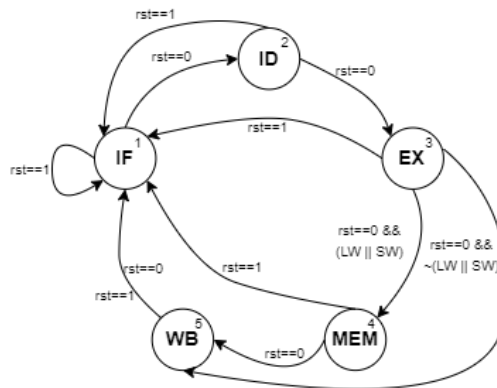
```
always @(instr)
begin
    if ((instr[6:0]==7'b0000011 && instr[14:12]==3'b010) ||
(instr[6:0]==7'b0100011 && instr[14:12]==3'b010) || (instr[6:0]==7'b0010011 &&
(instr[14:12]!=3'b001 && instr[14:12]!=3'b011 && instr[14:12]!=3'b101)))
        ALUSrc = 1'b1;
    else
        ALUSrc = 1'b0;
end
```

κατάλληλη τιμή από την προηγούμενη κιόλας κατάσταση ώστε να γίνει έγκαιρα η αλλαγή στη τιμή του PC. Η δεύτερη κατάσταση είναι η ID ενώ τη τρίτη η EX. Καμία από τις δύο δεν επηρεάζεται άμεσα από κάποιο σήμα ελέγχου. Τέταρτη κατάσταση

Πλαίσιο 10: Multicycle - Καθορισμός "PCSrc"

```
always @(instr,Zero)
begin
  if (instr[6:0]==7'b1100011 && instr[14:12]==3'b000)
    Branch = 1'b1;
  else
    Branch = 1'b0;
  PCSrc = Branch & Zero;
end
```

Διάγραμμα 4: Διάγραμμα - FSM



είναι η MEM για την οποία και ορίζονται δύο σήματα ελέγχου, “MemRead” και “MemWrite”, για την διεπαφή με την μνήμη δεδομένων. Το πρώτο χρησιμοποιείται για εντολές load ενώ το τελευταίο για εντολές store. Αν δεν ενεργοποιείται κανένα από τα δύο, δηλαδή δεν εκτελείται ούτε εντολή load ούτε store, τότε η κατάσταση αυτή μπορεί να παραλειφθεί. Είναι σημαντικό να τονισθεί ότι τα σήματα αυτά ενεργοποιούνται μονάχα στην κατάσταση MEM και για συγκεκριμένο τύπο εντολών όπως αυτές προσδιορίζονται μέσω των “opcode” και “funct3” στον Πίνακα 2. Πέμπτη και τελευταία κατάσταση είναι η WB για την οποία ορίζονται δύο σήματα ελέγχου, “MemToReg” και “RegWrite”. Το πρώτο σήμα χρησιμοποιείται για μία εντολή load και χρησιμεύει αποκλειστικά για την κατάσταση αυτή. Παρόλα αυτά η τιμή του δεν επηρεάζει καθόλου τις υπόλοιπες καταστάσεις του FSM. Το τελευταίο αφορά τις εντολές που γράφουν πίσω στο αρχείο καταχωρητών. Οι εντολές που γράφουν πίσω στο αρχείο καταχωρητών είναι εκείνες που έχουν στο πεδίο “instr[11:7]” την τιμή rd (destination register), δηλαδή όλες εκτός των BEQ και SW. Όλες οι εντολές αυτές έχουν προσδιοριστεί πλήρως στον Πίνακα 2. Τέλος, πρέπει να επισημανθεί πως το σήμα “RegWrite” ενεργοποιείται αποκλειστικά στην κατάσταση WB, ενώ παραμένει απενεργοποιημένο σε όλες τις υπόλοιπες καταστάσεις.

Όσον αφορά το κομμάτι Verilog η υλοποίηση του FSM χωρίζεται σε τρία procedural blocks που αφορούν την αποθήκευση της τρέχουσας κατάστασης, την λογική που προσδιορίζει την επόμενη κατάσταση και την λογική που προσδιορίζει τις τιμές των σημάτων ελέγχου ανάλογα με την εκάστοτε κατάσταση. Όπως έχει ήδη αναφερθεί παρατηρούνται πέντε διακριτές καταστάσεις οι οποίες κωδικοποιούνται σε μορφή “one hot”. Η κωδικοποίηση των καταστάσεων καθώς και το block που εκτελεί τη αποθήκευση της εκάστοτε τρέχουσας κατάστασης παρουσιάζονται στο Πλαίσιο 11. Η λογική επόμενης κατάστασης έχει ήδη αναφερθεί καθώς παρουσιάζεται σχηματικά

Πλαίσιο 11: Multicycle - Αποθήκευση Τρέχουσας Κατάστασης FSM

```
reg [4:0] current_state,next_state;
parameter instr_fetch = 5'b00001,instr_dec = 5'b00010,exec = 5'b00100,mem =
5'b01000,wr_back = 5'b10000;
always @(posedge clk or posedge rst)
begin: STATE_MEMORY
    if(rst)
        current_state <= instr_fetch;
    else
        current_state <= next_state;
end
```

Πλαίσιο 12: Multicycle - Λογική Επόμενης Κατάστασης FSM

```
always @(current_state)
begin: NEXT_STATE_LOGIC
    case (current_state)
        instr_fetch: next_state = instr_dec;
        instr_dec: next_state = exec;
        exec:
            begin
                if ((instr[6:0]==7'b0000011 || instr[6:0]==7'b0100011) &&
instr[14:12]==3'b010)
                    next_state = mem;
                else
                    next_state = wr_back;
            end
        mem: next_state = wr_back;
        wr_back: next_state = instr_fetch;
        default: next_state = instr_fetch;
    endcase
end
```

και στο Διάγραμμα 4 με τη χρήση αριθμών. Η υλοποίηση της σε Verilog παρουσιάζεται στο Πλαίσιο 12. Τέλος ακολουθεί, στο Πλαίσιο 13, η λογική ανανέωσης των σημάτων ελέγχου όπως ακριβώς αναλύθηκε στην προηγούμενη παράγραφο.

Το μόνο που απομένει για το module αυτό είναι η αρχικοποίηση του datapath που γίνεται απλά με την δημιουργία ενός στιγμιότυπου του module “datapath”. Η διασύνδεση των δοσμένων μνημών εντολών και δεδομένων, module “INSTRUCTION_MEMORY” και “DATA_MEMORY”, γίνεται με αντίστοιχο τρόπο, μετέπειτα, στο testbench.

Το μόνο που αξίζει να διευκρινιστεί είναι, όσον αφορά το testbench και τον καθορισμό των στιγμιότυπων του εκάστοτε module, είναι το σήμα “we” (write enable) του module “DATA_MEMORY”. Για το σήμα αυτό χρησιμοποιήθηκε η λογική “MemWrite & ~MemRead” που παρουσιάζεται στον Πίνακα 3. Στόχος της είναι το “we” να ενεργοποιείται μόνο σε μία εντολή τύπου store, συνδυάζοντας παράλληλα και τα δύο αυτά σήματα ελέγχου. Βέβαια, υπάρχουν και εντολές για τις οποίες τα δύο σήματα αυτά ταυτόχρονα απενεργοποιημένα. Τότε, σύμφωνα με την προαναφερθείσα λογική το “we” είναι απενεργοποιημένο, ώστε να μην υποδεικνύει κατάσταση εγγραφής. Άλλωστε, όπως υποδεικνύεται και στο Διάγραμμα 4, όταν δεν εκτελείται

Πλαίσιο 13: Multicycle - Λογική Ανανέωσης Σημάτων Ελέγχου FSM

```
always @(current_state)
begin: OUTPUT_LOGIC
case (current_state)
instr_fetch:
LoadPC = 1'b0;MemRead = 1'b0;MemWrite = 1'b0;MemToReg = 1'b0;RegWrite =
1'b0;
instr_dec:
LoadPC = 1'b0;MemRead = 1'b0;MemWrite = 1'b0;MemToReg = 1'b0;RegWrite =
1'b0;
exec:
begin
LoadPC = 1'b0;MemToReg = 1'b0;RegWrite = 1'b0;
if (instr[6:0]==7'b0000011 && instr[14:12]==3'b010)
MemRead <= 1'b1;MemWrite <= 1'b0;
else if (instr[6:0]==7'b0100011 && instr[14:12]==3'b010)
MemRead <= 1'b0;MemWrite <= 1'b1;
else
MemRead <= 1'b0;MemWrite <= 1'b0;
end
mem:
begin
LoadPC = 1'b0;MemRead <= 1'b0;MemWrite <= 1'b0;
if ((instr[6:0]==7'b0010011 && instr[14:12]!=3'b011) ||
(instr[6:0]==7'b0110011 && instr[14:12]!=3'b011))
MemToReg <= 1'b0;RegWrite <= 1'b1;
else if (instr[6:0]==7'b0000011 && instr[14:12]==3'b010)
MemToReg <= 1'b1;RegWrite <= 1'b1;
else
MemToReg <= 1'b0;RegWrite <= 1'b0;
end
wr_back:
LoadPC = 1'b1;MemRead = 1'b0;MemWrite = 1'b0;MemToReg <= 1'b0;RegWrite <=
1'b0;
default:
LoadPC = 1'b1;MemRead = 1'b0;MemWrite = 1'b0;MemToReg = 1'b0;RegWrite =
1'b0;
endcase
end
```

κάποια εντολή εγγραφής ή ανάγνωσης, το σύστημα φροντίζει να παραβλέπει το στάδιο MEM, καθιστώντας το σήμα “we” μη ουσιαστικής σημασίας στις περιπτώσεις αυτές.

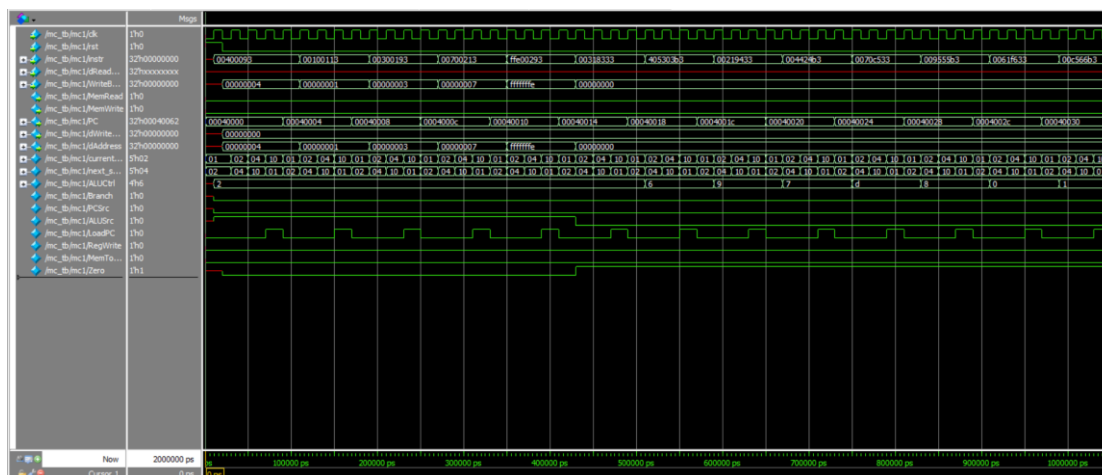
Τα Διαγράμματα 5 & 6 αποτελούν τα διαγράμματα χρονισμού του κυκλώματος. Το πρώτο αφορά την χρονική στιγμή από 0μs έως και 1μs ενώ το δεύτερο την χρονική στιγμή από 1μs έως 2μs.

Τέλος, ακολουθεί μία παράγραφος στην οποία παρουσιάζεται μία ελαφρώς διαφορετική προσέγγιση του προβλήματος, η οποία παρακάμπτει ορισμένες καταστάσεις σε περιπτώσεις που αυτές κρίνονται μη απαραίτητες.

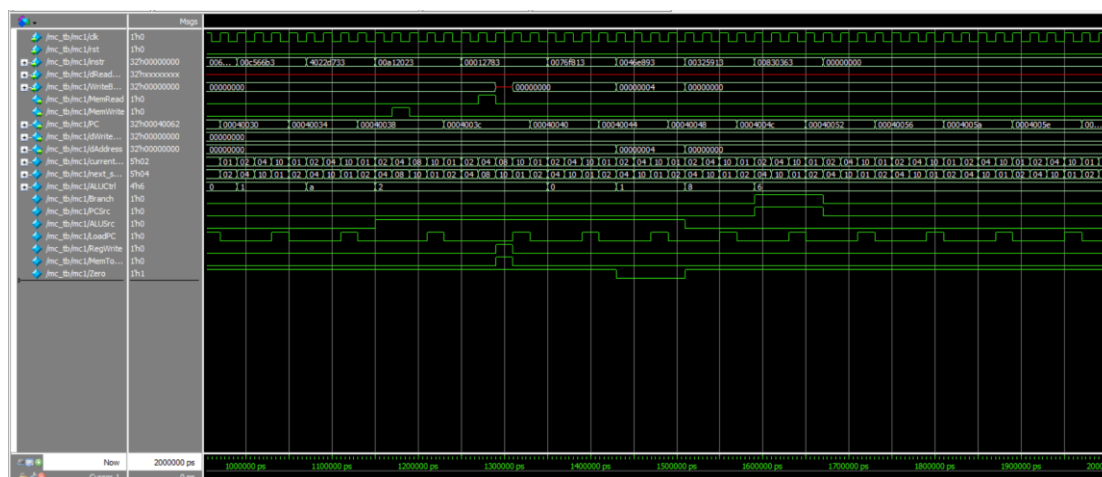
Πίνακας 3: Λογική "MemWrite & ~MemRead"

MemWrite	MemRead	MemWrite & ~MemRead
0	0	0
0	1	0
1	0	1
1	1	0

Διάγραμμα 5: Διάγραμμα Χρονισμού - RICS-V (0μs-1μs)



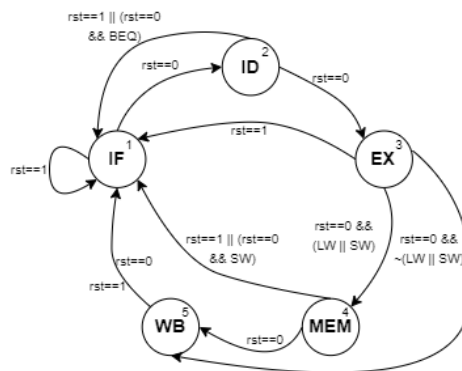
Διάγραμμα 6: Διάγραμμα Χρονισμού - RISC-V (1μs-2μs)



Άσκηση 5 - Alternate Version

Στην ενότητα αυτή παρουσιάζεται, συνοπτικά, μία ελαφρώς τροποποιημένη έκδοση του παραπάνω συστήματος. Έγινε η παρατήρηση πως κατά την εκτέλεση των εντολών SW και BEQ μπορούν να παραλειφθούν ορισμένες καταστάσεις με σκοπό την βελτιστοποίηση του συστήματος. Συγκεκριμένα, για την εντολή SW μπορεί να παραλειφθεί το στάδιο WB, καθώς η εντολή αυτή δεν γράφει τίποτα πίσω στο αρχείο καταχωρητών. Αντίστοιχα για την εντολή BEQ μπορούν να παραλειφθούν οι καταστάσεις EX, MEM, WB. Έτσι, το τροποποιημένο διάγραμμα της μηχανής πεπερασμένων καταστάσεων παρουσιάζεται στο Διάγραμμα 7.

Διάγραμμα 5: Διάγραμμα FSM (Alternate Version)



Πλαίσιο 14: Multicycle - Λογική Επόμενης Κατάστασης FSM (Alternate Version)

```

always @(current_state)
begin: NEXT_STATE_LOGIC
case (current_state)
instr_fetch: next_state = instr_dec;
instr_dec:
begin
if (instr[6:0]==7'b1100011 && instr[14:12]==3'b000)
next_state = instr_fetch;
else
next_state = exec;
end
exec:
begin
if ((instr[6:0]==7'b0000011 || instr[6:0]==7'b0100011) &&
instr[14:12]==3'b010)
next_state = mem;
else
next_state = wr_back;
end
mem:
begin
if (instr[6:0]==7'b0100011 && instr[14:12]==3'b010)
next_state = instr_fetch;
else
next_state = wr_back;
end
wr_back: next_state = instr_fetch;
default: next_state = instr_fetch;
endcase
end
  
```

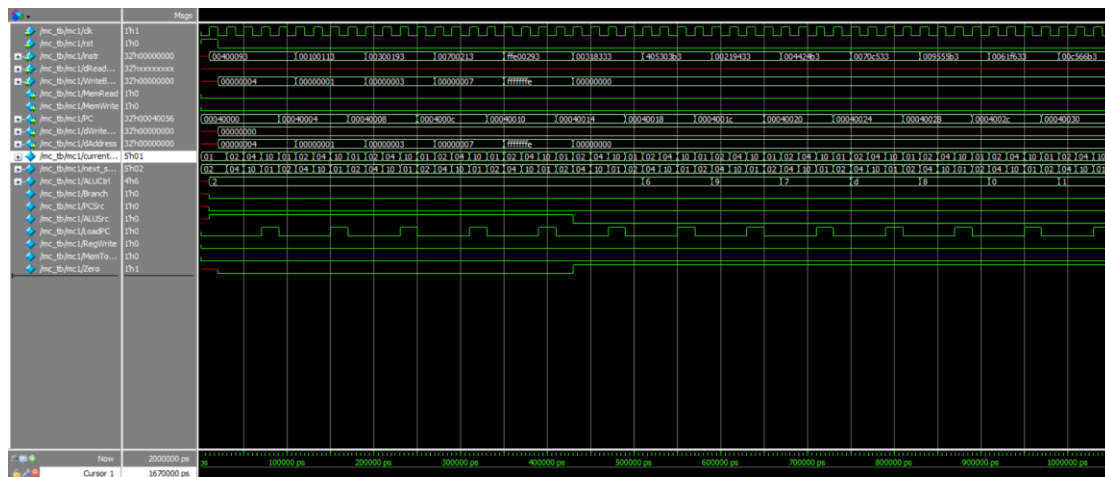
Πλαίσιο 15: Multicycle - Λογική Ανανέωσης Σημάτων Ελέγχου FSM (Alternate Version)

```
always @(current_state)
begin: OUTPUT_LOGIC
case (current_state)
instr_fetch:
LoadPC = 1'b0;MemRead = 1'b0;MemWrite = 1'b0;MemToReg = 1'b0;RegWrite =
1'b0;
instr_dec:
begin
MemRead = 1'b0;MemWrite = 1'b0;MemToReg = 1'b0;RegWrite = 1'b0;
if (instr[6:0]==7'b1100011 && instr[14:12]==3'b000)
LoadPC = 1'b1;
else
LoadPC = 1'b0;
end
exec:
begin
LoadPC = 1'b0;MemToReg = 1'b0;RegWrite = 1'b0;
if (instr[6:0]==7'b0000011 && instr[14:12]==3'b010)
MemRead <= 1'b1;MemWrite <= 1'b0;
else if (instr[6:0]==7'b0100011 && instr[14:12]==3'b010)
MemRead <= 1'b0;MemWrite <= 1'b1;
else
MemRead <= 1'b0;MemWrite <= 1'b0;
end
mem:
begin
LoadPC = 1'b0;MemRead <= 1'b0;MemWrite <= 1'b0;
if ((instr[6:0]==7'b0010011 && instr[14:12]!=3'b011) ||
(instr[6:0]==7'b0110011 && instr[14:12]!=3'b011))
MemToReg <= 1'b0;RegWrite <= 1'b1;
else if (instr[6:0]==7'b0000011 && instr[14:12]==3'b010)
MemToReg <= 1'b1;RegWrite <= 1'b1;
else if (instr[6:0]==7'b0100011 && instr[14:12]==3'b010)
MemToReg <= 1'b0;RegWrite <= 1'b0;LoadPC = 1'b1;
else
MemToReg <= 1'b0;RegWrite <= 1'b0;
end
wr_back:
LoadPC = 1'b1;MemRead = 1'b0;MemWrite = 1'b0;MemToReg <= 1'b0;RegWrite <=
1'b0;
default:
LoadPC = 1'b1;MemRead = 1'b0;MemWrite = 1'b0;MemToReg = 1'b0;RegWrite =
1'b0;
endcase
end
```

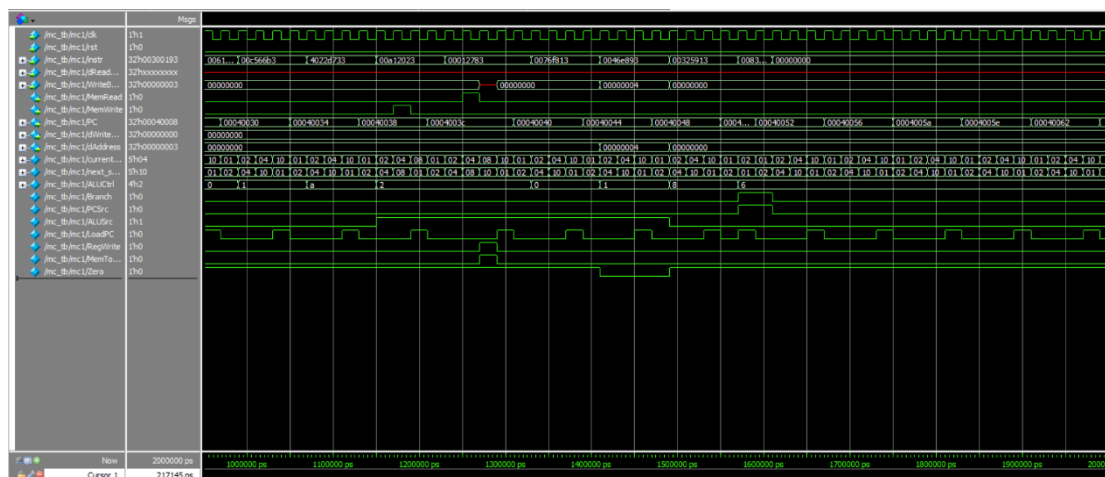
Όσον αφορά τον κώδικα οι μοναδικές αλλαγές που πρέπει να γίνουν αφορούν τα procedural block του FSM και συγκεκριμένα εκείνα της λογικής ανανέωσης της επόμενης κατάστασης και της λογικής ανανέωσης των σημάτων ελέγχου των Πλαισίων 12 & 13. Τα ανανεωμένα block κώδικα παρουσιάζονται εκ νέου στα Πλαίσια 14 & 15.

Τέλος , στα Διαγράμματα 8 & 9 παρουσιάζονται τα διαγράμματα χρονισμού του κυκλώματος, με την ίδια ακριβώς λογική, για χρόνο από 0μs έως 1μs και από 1μs έως 2μs.

Διάγραμμα 8: Διάγραμμα Χρονισμού - RICS-V (0μs-1μs)



Διάγραμμα 9: Διάγραμμα Χρονισμού - RISC-V (1μs-2μs)



Προβλήματα

Το αρχικό πλάνο ήταν να γίνει η εργασία μέσω της εφαρμογής “Questa Intel FPGA Starter Edition”. Ενώ είχε, όμως, ακολουθηθεί λεπτομερώς η διαδικασία απόκτησης της άδειας και εγκατάστασης της εφαρμογής δεν σταμάτησε να εμφανίζεται ποτέ το εξής πρόβλημα: “Fatal License Error: Unable to checkout a license. Make sure your license file environment variable (e.g., LM_LICENSE_FILE) is set correctly and then run ‘lmutil lmdiag’ to diagnose the problem”. Όμως, η μεταβλητή περιβάλλοντος είχε δημιουργηθεί κανονικά. Τρέχοντας στη γραμμή εντολών την εντολή ‘lmutil lmdiag’ εμφανιζόταν διαρκώς το μήνυμα: “lmutil - Copyright (c) 1989-2019 Flexera. All Rights Reserved”. Στο σημείο αυτό να διευκρινίσω ότι είχα δηλώσει το μάθημα και πέρυσι και πως ίσως επειδή είχα ήδη λάβει μία άδεια η οποία είχε λήξει να μπλοκαρίστηκε με κάποιο τρόπο η όλη διαδικασία. Βέβαια, η ανανέωση της άδειας έγινε κανονικά μέσα από τον επίσημο ιστότοπο, ακολουθώντας βήμα προς βήμα την προβλεπόμενη διαδικασία, καθώς έγινε αντίστοιχα και η ανανέωση της μεταβλητής περιβάλλοντος. Δεν κατάφερα, εν τέλη, να βγάλω κάποια άκρη με το συγκεκριμένο εργαλείο, οπότε και προχώρησα με το διαδικτυακό εργαλείο “EDA Playground”. Το τελευταίο κατάφερε κανονικά να τρέξει το testbench για την δεύτερη άσκηση αλλά εμφάνιζε αδιευκρίνιστα προβλήματα όσον αφορά το testbench της πέμπτης άσκησης. Τα διαγράμματα χρονισμού για την πέμπτη άσκηση τα έβγαλα με την βοήθεια συναδέλφου που δεν είχε κάποιο πρόβλημα με την εγκατάσταση του “Questa”, για αυτό και τα δύο διαγράμματα χρονισμού (δεύτερη και πέμπτη άσκηση) είναι διαφορετικά. Όλα αυτά τα καταγράφω ως μία απλή αναφορά, σε περίπτωση που αντιμετωπίζουν και υπόλοιποι συνάδελφοι κάποιο αντίστοιχο πρόβλημα.